Software Configuration Management (SCM):

Software Configuration Management is a **disciplined and systematic process** used to control, organize, and monitor changes to software and its related documents throughout the Software Development Life Cycle, from initial requirement analysis to final deployment and maintenance.

In real projects, software is not a single file. It is a collection of many related items that continuously change. SCM ensures that these items remain **consistent, traceable, and approved** at every stage.

Software Artifacts Managed Under SCM

SCM covers all items that define the software system, including:

- **Source code**
  - Program files written in languages such as Java, Python, or C++.
- **Requirement documents**
  - Software Requirement Specification documents that describe what the system should do.
- **Design diagrams**
  - UML diagrams, architecture diagrams, and database designs.
- **Test cases**
  - Test plans, test scripts, and test reports.
- **Deployment files**
  - Configuration files, environment settings, and installation scripts.
- **User manuals**
  - End-user guides and technical documentation.

Why SCM Is Essential

Without SCM:

- Developers may overwrite each other's work.
- Old or incorrect versions may be deployed.
- Bugs may reappear due to unmanaged rollbacks.
- Accountability is lost.

SCM ensures that **the right version of the right product is delivered to the right user at the right time**, with full traceability.

Objectives:

1. Maintain Consistency of Software Products SCM ensures that all components of the software match each other. **Example** If version 2.0 of source code is released, the requirement document, test cases, and user manual must also correspond to version 2.0.	**2. Control Unauthorized Changes** SCM prevents changes that are not reviewed or approved. **Example** A junior developer cannot directly modify production code without approval from the project manager or technical lead.
3. Track Evolution of Software SCM records: • Who made a change • What was changed • When it was changed • Why it was changed This creates a complete history of the software. **Example** If a bug appears in the current version, developers can trace back to the exact commit where the issue was introduced.	**4. Support Teamwork and Parallel Development** Multiple developers can work on different features at the same time without conflict. **Example** One team member works on billing, another on reporting, and another on authentication, all using controlled versions.
5. Improve Software Quality and Reliability Controlled changes reduce errors, duplication, and unexpected failures. **Example** Only tested and approved modules are allowed into the final release, improving stability.	

SCM Activities:

1. Configuration Identification

Meaning	**Typical Configuration Items**
Configuration Identification is the process of **selecting and naming the items** that need to be controlled	• Source code files • Requirement documents • Design diagrams • Test plans

These items are called **Configuration Items (CIs)**.	• Release packages Each CI is given: • A unique name • A version number
Example In a payroll system: • `Payroll_Requirements_v1.0` • `Salary_Calculation_Module_v2.1` • `Payroll_TestCases_v1.3` This avoids confusion between similar files.	**Case Study: Academic Project** In a final-year project, students failed to label document versions properly. As a result, outdated diagrams were used during implementation, causing rework. After adopting configuration identification rules, confusion was eliminated.

2. Configuration Control

Meaning Configuration Control ensures that **no change is made without proper approval**. Changes are handled through a formal process.	**Change Control Steps** 1. Change request submitted 2. Impact analysis performed 3. Approval or rejection 4. Implementation 5. Documentation update
Example A request to add fingerprint login is raised. The project manager evaluates cost and time before approving the change. **Case Study: Hospital Management System** A hospital requested a last-minute change to prescription formatting. Without configuration control, developers directly modified the system, causing system downtime. After introducing approval procedures, changes were scheduled and tested properly.	

3. Configuration Status Accounting

Meaning	**Example**
Configuration Status Accounting involves **recording and reporting the current status** of configuration items. It answers: • Which version is current? • What changes are pending? • What has been approved?	A status report may show: • Module A: Version 1.2 (Approved) • Module B: Version 1.1 (Under Testing) • Module C: Version 1.0 (Deprecated)

Case Study: Software Company

During an audit, a company could not explain which version was deployed at a client site. After implementing status accounting, detailed version reports were maintained, improving transparency and trust.

4. Configuration Auditing

Meaning	**Types of Audits**
Configuration Auditing checks whether: • Approved procedures were followed • Documentation matches the delivered software • Standards are met	• **Functional audit** ○ Verifies that the software meets requirements. • **Physical audit** • Ensures correct versions of files are delivered.

Example

Before release, auditors verify that the code version matches the approved requirement document and test results.

Case Study: Banking Software

A bank performs strict configuration audits before every release to ensure compliance with security and legal standards. Any mismatch leads to rejection.

Comprehensive Case Study: Hospital Management System

Problem Without SCM	SCM Implementation	Outcome
• Multiple versions of prescription modules • Old modules deployed by mistake • No record of changes • Increased medical risk	• Configuration items identified • Git used for version control • Change requests approved by a committee • Status reports maintained • Audits conducted before deployment	• Only approved prescription modules deployed • Reduced medical errors • Improved accountability • Higher system reliability

Version Control Systems (VCS):

A Version Control System is a software mechanism that **records, manages, and preserves changes** made to files over time. It allows developers to view earlier versions, compare changes, and safely coordinate work among multiple contributors.

In software projects, files change continuously. Without version control, teams face overwritten work, missing updates, and confusion about which file is correct. A VCS creates a **single source of truth** with a complete history.

Why Version Control Is Essential in Software Projects

1. **Multiple contributors** work on the same files
2. **Frequent updates** are required due to bugs and new requirements
3. **Rollback** is needed when errors are introduced
4. **Accountability** is required for audits and maintenance

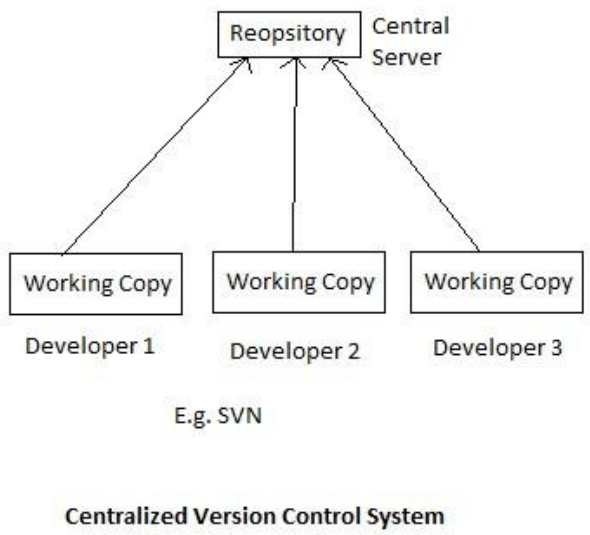
Version control ensures **control without blocking progress**.

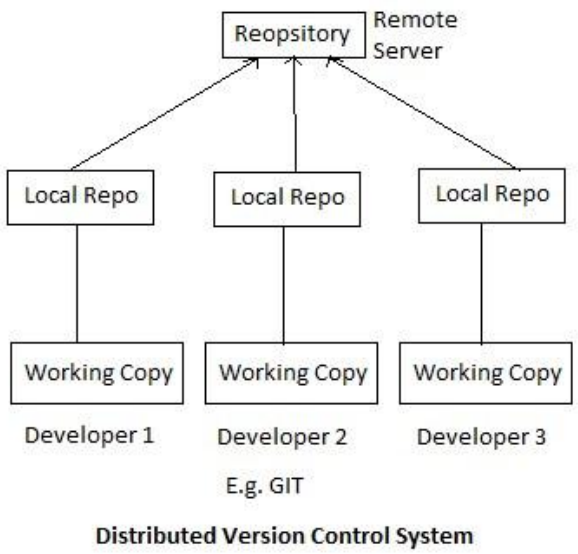
Core Concepts in Version Control

Repository	Commit
A repository is a storage location that contains: • Project files • Version history	A commit is a **snapshot of changes** saved into the repository.

• Metadata about changes Repositories may be local, centralized, or distributed.	Each commit usually includes: • What changed • Who changed it • When it was changed • Why it was changed
Branch A branch is a separate line of development that allows work to proceed independently.	**Merge** Merging combines changes from one branch into another.
Revert / Rollback Restores files to an earlier, stable version.	

Types of Version Control Systems





7.2.1 Centralized Version Control System (CVCS)

Concept In a centralized system, there is **one central repository** stored on a server. All developers connect to this server to: • Download files • Upload changes	**Characteristics** • Single point of control • Simple structure • Easy to manage access rights

If the server is unavailable, work stops.	
Characteristics • Single point of control • Simple structure • Easy to manage access rights	**Advantages** • Centralized administration • Clear control over codebase • Suitable for small, controlled teams
Limitations • Server failure halts development • Limited offline work • Slower for large teams	**Example** A small government IT department uses a centralized repository hosted on an internal server. Developers must be connected to the office network to commit changes.

Case Study: Government Payroll System

Scenario
A payroll application is developed for a public office. All code must remain within government infrastructure for security reasons.

Why Centralized VCS Works • Code never leaves the server • Access is strictly controlled • All changes are reviewed centrally	**Outcome** Security is maintained, but developers face delays during network outages.

7.2.2 Distributed Version Control System (DVCS)

Concept In a distributed system, **every developer has a full copy** of the repository, including complete history. Developers can work independently and synchronize changes later.	**Characteristics** • No single point of failure • Full offline capability • Strong support for branching and merging
Advantages • Faster operations • High reliability • Better collaboration • Encourages experimentation	**Limitations** • Slightly higher learning curve • Requires disciplined workflow

Example

A university software lab project uses a distributed system. Students work from home, make commits locally, and synchronize when internet access is available.

Detailed Working of Git (DVCS)

Repository Structure	Typical Git Workflow
• **Local repository**: Developer's personal copy • **Remote repository**: Shared central location for synchronization	1. Clone the repository 2. Create a branch 3. Make changes 4. Commit locally 5. Merge and push to the shared repository

Example

A developer adds a login feature in a separate branch. After testing, the branch is merged into the main branch without affecting other features.

Case Study: E-Commerce Website Development

Scenario	Problems Without Version Control
An e-commerce website is being developed by a team of eight developers.	• Files overwritten • Conflicting updates • Lost code
Solution Using Git	**Outcome**
• Each feature developed in separate branches • Code reviewed before merging • Bugs traced to specific commits	• Faster development • Fewer integration errors • Clear accountability

Version Control in Academic Projects

Example: Final Year Project

Without VCS	With VCS
• Files shared via pen drive or email • Confusion over latest version • Accidental deletion	• Clear history of contributions • Easy rollback • Fair evaluation of student work

Case Study: University Management System

Scenario
A team develops modules for attendance, grading, and finance.

Implementation	Outcome
• Separate branches for each module • Weekly merges • Tagged releases	• Smooth integration • Minimal conflicts • Timely submission

Role of Version Control in Maintenance

Version control is not limited to development. It is critical during:

- Bug fixing
- Feature enhancement
- Security patching

Example

A bug appears after deployment. Developers trace the issue to a specific commit and revert only that change instead of rewriting the entire code.

Centralized Version Control System: SVN

Meaning of SVN (Subversion)

SVN is a **Centralized Version Control System** where:

- A single central repository exists on a server

- All developers check out files from this server
- All commits are sent back to the same server

If the server is unavailable, work is disrupted.

Working of SVN	**Characteristics of SVN**
1. Developer checks out files from the server 2. Makes changes locally 3. Commits changes back to the server 4. Other developers update their copies	• Central server controls the codebase • Requires network connectivity • Linear version history • Simple branching and merging
Advantages of SVN	**Limitations of SVN**
• Centralized control and security • Easy access management • Simple concept for beginners • Suitable for regulated environments	• Single point of failure • Limited offline work • Slower for large teams • Branching is less flexible

Example of SVN Use

A government office maintains a centralized server where all payroll software code is stored. Developers can access and modify code only within office hours and network.

Case Study: Government Payroll System Using SVN

Context
A payroll system for civil servants must comply with strict data security rules.

Why SVN Was Chosen	**Outcome**
• Code must remain inside government servers • Access must be centrally controlled • Audit trail required	• Strong control and security • Slower development during network outages • Clear accountability during audits

Distributed Version Control System: Git

Meaning of Git

Git is a **Distributed Version Control System** where:

- Every developer has a complete copy of the repository
- Full history is stored locally
- Developers can work offline
- Changes are synchronized later

Git is the most widely used version control system in the software industry.

Working of Git	**Key Features of Git**
1. Clone the repository 2. Create a branch 3. Modify files 4. Commit changes locally 5. Merge branches 6. Push changes to shared repository	• Fast performance • Powerful branching and merging • Offline commits • Strong community support
Advantages of Git	**Limitations of Git**
• No single point of failure • High speed • Encourages experimentation • Ideal for Agile and DevOps	• Learning curve for beginners • Requires disciplined workflow • Poor practices can cause merge conflicts

Example of Git Use

A team developing a mobile application uses Git. One developer works on login features, another on payment processing. Both work independently and merge changes after testing.

Case Study: E-Commerce Website Development Using Git

Context
An e-commerce website is built by a distributed team working from different locations.

Problems Before Git	**Git-Based Solution**	**Outcome**
• Code conflicts • Delayed integration	• Feature-based branches	• Faster development • Reduced bugs

• Lost changes	• Code reviews before merge • Tagged releases	• Clear version history

Configuration Management Plan (CMP)

A **Configuration Management Plan (CMP)** is a **formal, documented guideline** that defines **how Software Configuration Management will be implemented and controlled** throughout a software project.

In simple terms, CMP answers this question:

How will we identify, control, track, and audit changes to all software-related items during the project?

The CMP acts as a **rulebook** for managing versions, changes, responsibilities, and tools.

Why it Is Required?

Software projects involve:	Without a CMP:
• Multiple developers • Frequent changes • Many documents and files • Long development timelines	• Different versions get mixed • Unauthorized changes occur • Responsibility becomes unclear • Releases become unstable

A CMP brings **discipline, predictability, and traceability**.

Objectives:

1. **Define configuration items clearly**
2. **Establish change control procedures**
3. **Assign roles and responsibilities**
4. **Standardize versioning and naming rules**
5. **Ensure auditability and compliance**
6. **Reduce errors caused by uncontrolled changes**

When CMP Is Prepared	Scope of CMP
• Prepared during the **project planning phase** • Approved before development starts	The CMP applies to: • Entire project life cycle

• Updated if project scope or tools change	• Development, testing, deployment, and maintenance • All software and documentation artifacts

Major Components of a Configuration Management Plan

1. Introduction and Purpose	Example
Description This section explains: • Why the CMP exists • What project it applies to • What it aims to achieve	"This Configuration Management Plan defines the procedures for identifying, controlling, and auditing configuration items for the Hospital Management System project."

2. Configuration Items (CI) Identification

Meaning	Types of Configuration Items	Each CI must have:
Configuration Items are **any artifacts that must be controlled** because changes to them affect the project.	• Source code files • Requirement documents • Design documents • Test cases and reports • Deployment scripts • User manuals	• Unique name • Version number • Owner

Example

Configuration Item	Type	Version
HMS_Requirements	Document	v1.0
Patient_Module	Source Code	v2.3
Test_Report	Test Document	v1.2

Case Study: Academic Project Failure

Students did not define configuration items. Multiple versions of reports and code were submitted, leading to confusion and poor evaluation. Introducing CI identification solved the problem.

3. Version Control Strategy

Purpose	Versioning Rules	Example
Defines how versions will be created, named, and managed.	• Major version change for major features • Minor version change for small enhancements • Patch version for bug fixes	• Version 1.0: Initial release • Version 1.1: Feature enhancement • Version 1.1.1: Bug fix

Example in Practice

Only stable versions are tagged as release versions. Experimental changes remain in development branches.

4. Change Control Process

Meaning	Change Request Process	Change Request Form Typically Includes
Change control ensures **no change is made without proper review and approval**.	1. Change request submitted 2. Impact analysis performed 3. Approval or rejection 4. Implementation 5. Documentation update	• Change description • Reason for change • Impact on cost and schedule • Priority level

Case Study: Hospital Management System

A request to modify prescription format was reviewed for medical safety, testing effort, and training impact before approval. This prevented unsafe deployment.

5. Roles and Responsibilities

Key Roles in CMP	Example
• **Project Manager** Approves changes and releases • **Configuration Manager** Maintains repositories and records • **Developers** Implement approved changes • **Quality Assurance Team** Verifies correctness and compliance	Only the configuration manager is allowed to merge changes into the main branch.

6. Configuration Status Accounting

Meaning	Example Status Report
Configuration Status Accounting records and reports: • Current version of each CI • Approved and pending changes • Release history	

Item	Version	Status
Login Module	v1.2	Approved
Payment Module	v1.3	Under Testing
Reports Module	v1.1	Deprecated

Case Study

A software firm failed an audit because it could not show which version was delivered. After implementing status accounting, audits were cleared successfully.

7. Configuration Auditing

Meaning	Types of Audits	Example
Configuration Auditing ensures: • Changes follow approved procedures • Delivered software matches documentation	• **Functional Audit** Checks whether requirements are satisfied • **Physical Audit** Verifies correct versions are included in the release	Before release, auditors verify that source code version matches approved requirements and test reports.

• Standards are followed		

8. Tools and Technologies

Common Tools Mentioned in CMP	Example
• Version control tools like Git or SVN • Issue tracking tools • Build and deployment tools	"Git will be used for source code management. JIRA will be used for change request tracking."

9. Access Control and Security

Purpose	Example
Defines who can: • View files • Modify files • Approve changes	• Developers can modify code • Test team can view code • Only project manager can approve releases

10. Release Management Interface

Description

Defines how SCM interacts with release management:

- Which versions can be released
- Release approval authority
- Release documentation

Comprehensive Case Study: Banking Software Project

Without CMP	With CMP
• Multiple versions deployed • Unauthorized changes • Failed audits	• Clear configuration items • Approved change control • Audited releases • Legal compliance

Advantages of Using CMP	Limitations of CMP
• Improved project control • Reduced rework • Clear accountability • Better quality • Audit readiness	• Requires documentation effort • Needs discipline and training • May seem slow for very small projects

Change Management

Change Management is a **formal and systematic process** used to **identify, evaluate, approve, implement, and document changes** in a software project.

In software projects, change is unavoidable. Requirements evolve, users request new features, bugs are discovered, and regulations change. Change Management ensures that such changes are **handled in a controlled way**, without harming cost, schedule, quality, or stability.

In simple terms:

Change Management decides **what can be changed, when it can be changed, and how it will be changed**.

Why Change Management Is Necessary?

Without proper change management:

- Project scope keeps expanding without control
- Cost and schedule overrun
- Quality degrades
- Team confusion increases
- Clients lose trust

Change Management protects the project from **chaos caused by uncontrolled modifications**.

Sources of Change in Software Projects	Objectives of Change Management
Changes may arise from: 1. **Client or user requests** New features, UI changes, reports, language support	• Control project scope • Minimize negative impact of changes • Ensure informed decision-making • Maintain documentation accuracy

2. **Technical issues** Performance problems, security vulnerabilities 3. **Bug fixes** Errors discovered during testing or after deployment 4. **Business or policy changes** New laws, compliance requirements, organizational rules 5. **Technology changes** Framework updates, database migration	• Improve customer satisfaction

Types of Changes

1. Functional Changes Changes to system features or behavior. **Example** Adding OTP-based login instead of simple password login.	**2. Non-Functional Changes** Changes related to performance, security, usability, or reliability. **Example** Improving response time of a search module.	**3. Corrective Changes** Fixing defects or errors. **Example** Fixing incorrect tax calculation logic.
4. Adaptive Changes Adjusting software to new environments or platforms. **Example** Updating software to support a new operating system.	**5. Preventive Changes** Changes made to avoid future problems. **Example** Refactoring code to reduce future maintenance issues.	

Change Management Process (Step-by-Step)

Step 1: Change Request Initiation A **Change Request (CR)** is formally submitted.	**Step 2: Change Impact Analysis** The proposed change is analyzed for its impact on:

A change request usually includes:

- Description of the change
- Reason for change
- Priority
- Requested by
- Expected benefits

Example

A hospital requests adding blood group information to patient reports.

- Cost
- Schedule
- Resources
- Risk
- Quality

This step answers:

Is the change feasible and worth implementing?

Example

Adding blood group data requires database changes, UI updates, testing, and staff training.

Step 3: Change Control Board (CCB) Review

A **Change Control Board** reviews the change request.

Members of CCB

- Project manager
- Technical lead
- Quality assurance representative
- Client or user representative

The CCB may:

- Approve the change
- Reject the change
- Defer the change

Example

The CCB approves the change but schedules it for the next release due to workload.

Step 4: Change Approval or Rejection

The decision is documented:

- Approved changes move forward
- Rejected changes are recorded with reasons

Example

A request to redesign the entire system UI late in the project is rejected due to high cost and delay.

Step 5: Change Implementation

Approved changes are:

- Assigned to developers

Step 6: Testing and Validation

Changes are tested to ensure:

- No new defects are introduced

• Implemented following coding standards • Integrated with existing system **Example** Developers update database schema and modify report templates.	• Existing functionality still works • Change meets requirements **Example** Regression testing is performed to ensure patient reports still generate correctly.
Step 7: Documentation Update All related documents are updated: • Requirements document • Design documents • User manuals • Release notes **Example** User manual is updated to explain the new blood group field.	**Step 8: Change Closure** The change is formally closed after: • Successful testing • Documentation update • Stakeholder confirmation

Case Study 1: Hospital Management System

Situation Without Change Management	Situation With Change Management	Outcome
• Doctors requested report changes verbally • Developers made direct code changes • Old reports stopped working • Patient safety risk increased	• Formal change requests submitted • Medical and technical impact analyzed • Approved changes tested thoroughly	• Safer system • Fewer errors • Improved trust from hospital staff

Case Study 2: Academic ERP System

Problem	Change Management Applied	Outcome
Students requested last-minute grade calculation changes.	• Change request logged • Impact on exams and deadlines analyzed • Approved for next semester release	• Current exams completed on time • New feature delivered later without disruption

Relationship Between Change Management and SCM

Change Management works **within Software Configuration Management**:

- SCM controls versions
- Change Management controls decisions
- CMP defines procedures
- Version control tools implement approved changes

Advantages of Change Management	Limitations of Change Management
• Prevents scope creep • Improves project predictability • Enhances quality • Maintains stakeholder confidence • Ensures audit readiness	• Requires documentation effort • Can slow rapid changes • Needs disciplined team behavior

Release Management

Release Management is the **process of planning, scheduling, controlling, and delivering approved changes** to users in the form of a software release.

While **Change Management decides what should change**, **Release Management decides when and how those changes are delivered** safely and reliably.

In simple words:

Release Management ensures that only tested, approved, and complete software versions are deployed to users.

Why Release Management Is Necessary

Without proper release management:

- Unstable software reaches users
- Frequent unplanned deployments occur
- Bugs reappear
- Users lose confidence
- Business operations are disrupted

Release Management ensures **stability, predictability, and quality** in software delivery.

Relationship with SCM and Change Management

- **SCM** controls versions and configuration items
- **Change Management** approves or rejects changes
- **Release Management** bundles approved changes and delivers them

All three work together to ensure controlled software evolution.

Objectives of Release Management

- Deliver stable and reliable software
- Minimize deployment risks
- Coordinate development, testing, and operations
- Ensure traceability of released versions
- Maintain release documentation

Key Concepts in Release Management

Release	**Baseline**	**Deployment**
A release is a **specific version of software** made available to users.	A baseline is a **stable reference version** from which further changes are made.	Deployment is the process of installing the release into the target environment.

Types of Software Releases

1. Major Release	2. Minor Release
Description Introduces significant new features or major changes. **Example** Version 3.0 of a university ERP system introducing online admission and payment modules.	**Description** Adds small enhancements or improvements without major changes. **Example** Version 2.1 adding new report formats.
3. Patch Release	**4. Emergency Release**
Description Fixes bugs or security issues. **Example** Version 2.1.1 fixing login errors.	**Description** A quick release to fix critical issues. **Example** Immediate security patch after vulnerability detection.

Release Management Process (Step-by-Step)

Step 1: Release Planning	Step 2: Release Build and Assembly
Description Defines: • What changes will be included • Release objectives • Target users • Release date Only **approved changes** from Change Management are considered. **Example**	**Description** Approved components are: • Integrated • Compiled • Packaged This forms a **release candidate**. **Example**

A hospital system plans a quarterly release including prescription updates and report enhancements.	Multiple modules developed separately are combined into a single deployment package.
Step 3: Testing and Validation **Types of Testing** • Unit testing • Integration testing • System testing • User Acceptance Testing Testing ensures that the release: • Meets requirements • Does not break existing functionality **Example** Before releasing a payroll system update, salary calculations are tested for all employee categories.	**Step 4: Release Approval** **Description** Final approval is given by: • Project manager • Quality assurance team • Client representative Only after approval can deployment proceed. **Example** A release is delayed by one week due to failed security testing.
Step 5: Deployment **Description** The release is deployed to: • Production environment • Client systems Deployment may be: • Manual • Automated **Example** An academic ERP system is deployed during semester break to avoid disruption.	**Step 6: Release Documentation** **Documentation Includes** • Release notes • Version number • New features • Fixed issues • Known limitations **Example** Release notes explain new grading features and known browser limitations.
Step 7: Post-Release Review	

Description After deployment: • System performance is monitored • User feedback is collected • Issues are logged **Example** User complaints after release help identify minor UI issues for the next update.

Case Study 1: Hospital Management System

Problem Without Release Management	**Release Management Introduced**	**Outcome**
• Developers deployed changes directly • Unstable versions used by doctors • Incorrect patient reports generated	• Quarterly planned releases • Mandatory testing • Approval before deployment	• Improved system reliability • Reduced medical risk • Increased user trust

Case Study 2: Banking Software

Context	**Release Management Approach**	**Outcome**
Banking software requires strict compliance and stability.	• Few releases per year • Extensive testing and audits • Rollback plans prepared	• Zero data loss • Regulatory compliance • High customer confidence

Role of Version Control in Release Management

Version control supports release management by:

- Tagging release versions
- Creating baselines

- Tracking changes included in a release

Example

Version 2.0 is tagged as a release point. Future changes start from this baseline.

Advantages of Release Management	Limitations of Release Management
• Predictable software delivery • Reduced deployment risk • Better coordination • Improved software quality • Clear accountability	• Requires planning effort • Slower delivery if poorly managed • Needs coordination across teams

Despite limitations, release management is essential for professional software development.

Overview of Project Management Tools

MS Project, JIRA, Trello

Project management tools help a team **plan work, assign responsibilities, track progress, manage risks/issues, and report status**. In software projects, these tools also support Agile boards, sprint planning, change tracking, and release visibility.

Microsoft Project (MS Project)

What it is used for?

MS Project is mainly used for **schedule-based project planning**. It is strong for traditional project management where you need clear timelines and task dependencies.

Key features

1. **Work Breakdown Structure (WBS) and task lists**
   Break the whole project into tasks and subtasks.
2. **Gantt Chart (timeline view)**
   Shows tasks on a timeline with start and finish dates.
3. **Task dependencies**
   Defines relationships like:
   - Finish-to-Start (Task B starts after Task A finishes)
   - Start-to-Start (both can start together)

4. **Resource assignment and workload**
   Assign people to tasks and check over-allocation.
5. **Milestones and critical path**
   Helps identify tasks that control the project end date.
6. **Baseline vs actual tracking**
   Set a baseline plan and compare it with actual progress.

Example (classroom-friendly)

Project: "Online Library System"

- Task: Requirement analysis (5 days)
- Task: UI design (7 days) depends on requirement analysis
- Task: Development (20 days) depends on UI design
  MS Project helps students visualize how delays in requirements affect the whole schedule.

Mini Case Study

Scenario: A campus ERP project has frequent delays.
Use of MS Project: The manager sets baseline dates and monitors variance weekly. The critical path shows testing is the main bottleneck, so extra testers are assigned.
Outcome: Delays reduce because the team focuses on schedule-critical tasks instead of random tasks.

Strengths	Limitations
• Best for schedules, dependencies, and timelines • Strong reporting for time management • Good for large, structured projects	• Not ideal for daily Agile task flow • Can feel heavy for small student teams • Needs careful updating to stay accurate

JIRA

What it is used for?

JIRA is widely used in software teams for **issue tracking and Agile project management**. It is excellent for managing:

- user stories
- tasks
- bugs
- sprints
- releases

Key features

1. **Backlog and sprint planning**
   - Create user stories
   - Prioritize backlog
   - Move items into a sprint
2. **Scrum and Kanban boards**
   Visual workflow: To Do → In Progress → Done
3. **Issue types**
   - Story (new feature)
   - Bug (defect)
   - Task (work item)
   - Epic (big feature group)
4. **Workflow and approvals**
   Custom steps like:
   - To Do → In Progress → Code Review → Testing → Done
5. **Reports and dashboards**
   - Burndown chart (sprint progress)
   - Velocity chart (team capacity)
   - Issue aging (delayed tasks)
6. **Release and version tracking**
   Mark issues that will go into Release 1.1, 1.2, etc.

Example

In a "Food Delivery App" project:

- Story: "User can register with email and phone"
- Bug: "OTP not received sometimes"
  JIRA helps assign stories and bugs to team members and track sprint completion.

Mini Case Study

Scenario: A software team keeps missing sprint targets.
Use of JIRA: The Scrum master tracks burndown chart daily, notices too many tasks stay "In Progress," and enforces smaller user stories plus daily review.
Outcome: Sprint predictability improves, and bug count reduces because testing is included inside the workflow.

Strengths	Limitations
• Excellent for Agile and software development • Strong bug tracking and reporting • Traceable change and release tracking	• Setup and workflow configuration takes time • Can feel complex for beginners • Requires discipline in updating status

Trello

What it is used for	Key features

Trello is a simple visual task management tool based on a **Kanban board**. It is popular for:
 	
• student projects
• small teams
• lightweight planning
	1. **Boards, lists, and cards**
	  o Board: Project
	  o Lists: To Do, Doing, Done
	  o Cards: Tasks
	2. **Checklists and due dates**
	Break a card into subtasks.
	3. **Labels and members**
	Labels for categories like UI, Backend, Testing.
	4. **Simple collaboration**
	Comments, attachments, notifications.

Example

Student project "Attendance System" board:

- To Do: Database design, Login UI
- Doing: QR scanner module
- Done: Requirement document

Mini Case Study

Scenario: A student group struggles to coordinate tasks.
Use of Trello: They create weekly task cards and assign members. Daily they move cards across lists.
Outcome: Everyone sees progress clearly, and tasks stop getting forgotten.

Strengths	Limitations
• Very easy to learn and use • Great for small teams and classroom projects • Visual and lightweight	• Limited advanced reporting • Not ideal for complex dependencies and baselines • Difficult to manage very large projects

Legal and Ethical Issues in Software Project Management

Software project management is not only about planning, coding, and delivery. Project managers are also responsible for ensuring that the project **follows the law and meets ethical standards**. Failure in these areas can lead to legal penalties, loss of reputation, and harm to users.

Legal Issues in Software Project Management

Meaning of Legal Issues

Legal issues refer to **laws, regulations, contracts, and rights** that must be followed during software development and delivery. A project manager must ensure compliance at every stage of the project life cycle.

1. Software Licensing Issues	2. Copyright and Intellectual Property Rights (IPR)
Explanation Software tools, libraries, and frameworks are protected by licenses. Using them without permission or violating license terms is illegal. **Common License Types** • Proprietary licenses • Open-source licenses • Freeware and shareware licenses **Example** Using a paid IDE or database without a valid license in a commercial project can lead to legal action. **Case Study** A startup used an open-source library that required source code disclosure but ignored the license terms. After release, they were forced to open their proprietary code, causing business loss.	**Explanation** Copyright protects original software code, documentation, designs, and content. Intellectual Property Rights define who owns the software. **Key Concerns** • Code plagiarism • Ownership disputes • Reuse of third-party code **Example** Copying code from the internet without permission or citation violates copyright laws. **Case Study** Two companies disputed ownership of a software product because the contract did not clearly define IP ownership. The case delayed product launch for months.

3. Contractual Obligations

Explanation

Software projects are governed by contracts that specify:

- Scope of work
- Payment terms
- Delivery deadlines
- Penalties for delay

Failure to meet contractual terms can result in legal disputes.

Example

If a project is delivered late without agreed extensions, the client may demand compensation.

4. Data Protection and Privacy Laws

Explanation

Software often handles personal and sensitive data. Laws require this data to be collected, stored, and processed securely.

Common Concerns

- Unauthorized data access
- Data leaks
- Improper data sharing

Example

Storing user passwords in plain text is a violation of data protection laws.

Case Study

A health application leaked patient data due to poor security, leading to legal penalties and loss of public trust.

5. Compliance with Industry Standards

Explanation

Certain domains require compliance with standards such as:

- Security standards
- Financial regulations
- Healthcare regulations

Example

Banking software must comply with security and audit requirements before deployment.

Ethical Issues in Software Project Management

Meaning of Ethical Issues

Ethical issues relate to **moral responsibility, fairness, honesty, and professional conduct**, even when no law is broken.

Ethics guide how decisions should be made to protect users, clients, and society.

1. Honesty and Transparency **Explanation** Project managers must report: • True project status • Realistic timelines • Actual risks **Example** Hiding delays to appear successful is unethical. **Case Study** A project manager falsely reported progress, leading to sudden project failure near deadline.	**2. Data Privacy and User Trust** **Explanation** Ethical responsibility goes beyond legal compliance. Users trust software with their data. **Example** Selling user data without consent, even if legal loopholes exist, is unethical.
3. Fair Treatment of Team Members **Explanation** Ethics require: • Fair workload distribution • Respectful behavior	**4. Avoiding Plagiarism** **Explanation** Using others' work without proper acknowledgment is unethical, even in academic projects. **Example**

• No discrimination **Example** Overburdening junior developers while seniors avoid responsibility is unethical.	Submitting copied project code as original work violates academic ethics.

5. Quality and Safety Responsibility

Explanation

Releasing knowingly defective software can harm users.

Example

Releasing a hospital system with known calculation errors risks patient safety.

Case Study

A software firm rushed a medical application to meet deadline, leading to incorrect dosage calculations. The ethical failure caused serious harm and legal consequences.

Role of Project Manager in Legal and Ethical Compliance

A project manager must:

- Understand relevant laws and licenses
- Ensure ethical decision-making
- Promote transparency
- Protect user interests
- Enforce professional standards

Relationship Between Legal and Ethical Issues

Legal	Ethical
Mandatory by law	Based on moral principles
Punishable if violated	Damages trust and reputation
Minimum requirement	Higher professional standard

Comprehensive Case Study: Educational Software Project

Problem	Impact	Ethical and Legal Resolution
A team used pirated software tools and copied code from online sources.	• Legal risk to institution • Academic misconduct • Loss of credibility	• Use licensed or open-source tools properly • Acknowledge sources • Train students on ethics

Advantages of Addressing Legal and Ethical Issues

- Avoid legal penalties
- Build user trust
- Improve software quality
- Enhance professional reputation